2025 International Conference on Cybersecurity and AI-Based Systems (Cyber-AI) | 979-8-3315-6631-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/Cyber-AI66431.2025.11233258

# ShadowPlay: Engineering Defenses Against Role-Based Prompt Injection and Dependency Hallucination in LLM-Powered Development

1st Anas AlSobeh
*Information Technology and Cybersecurity*
*Southern Illinois University*
Carbondale, IL 62901 USA
anas.alsobeh@siu.edu
orcid: 0000-0002-1506-7924

2nd Zahraddeen Gwarzo
*School of Computing*
*Weber State University*
Ogden, UT 84405 USA
zahraddeengwarzo1@weber.edu

3rd Amani Shatnawi
*School of Computing*
*Weber State University*
Ogden, UT 84405 USA
amanishatnawi1@weber.edu

***Abstract*—Large Language Models (LLMs) have revolutionized software development by providing intelligent code generation and assistance capabilities. However, their integration into development workflows introduces critical security vulnerabilities that traditional cybersecurity measures fail to address. This paper presents ShadowPlay, a novel multi-layered defense framework specifically engineered to detect and mitigate two emerging threat vectors: role-based prompt injection attacks and dependency hallucination vulnerabilities. Our approach combines formal verification techniques with Machine Learning (ML)-based behavioral analysis to create a comprehensive security solution. The framework employs four integrated components: a semantic prompt analysis engine, a real-time dependency verification system, a behavioral monitoring module, and a response validation layer. Through extensive evaluation on over 20,000 attack scenarios across diverse threat categories, ShadowPlay demonstrates superior performance with 95.1% overall accuracy, 97.2% precision, and only 1.8% false positive rate, while maintaining an average response latency of 127 milliseconds. Our contributions include the first formal mathematical model for these attack vectors, a production-ready defense architecture, and comprehensive empirical validation demonstrating significant improvements over existing approaches.**

***Index Terms*—Large Language Models, Prompt Injection, Software Security, ML Security, Dependency Management, Behavioral Analysis.**

## I. Introduction

The rapid adoption of Large Language Models in software development environments has fundamentally transformed how developers write, debug, and maintain code. Modern development workflows increasingly rely on LLM-powered assistants for tasks ranging from code generation and documentation to architectural design and security analysis. However, this integration introduces unprecedented security challenges that existing cybersecurity frameworks struggle to address effectively.

Recent studies have identified two critical vulnerability classes that pose significant threats to LLM-powered development environments. Role-based prompt injection attacks exploit the conversational nature of LLM interactions by manipulating the model's perceived authority context, leading to unauthorized actions or information disclosure. Dependency hallucination attacks target the model's tendency to suggest non-existent or malicious packages, potentially introducing supply chain vulnerabilities into software projects [1], [2].

Traditional security measures prove inadequate against these sophisticated attack vectors because they operate at the application layer rather than the semantic level where LLM interactions occur. Existing approaches typically rely on simple keyword filtering or rule-based systems that attackers can easily circumvent through linguistic manipulation or context switching. The dynamic and contextual nature of natural language interactions requires fundamentally different defensive strategies that can understand semantic intent and behavioral patterns.

This paper introduces ShadowPlay, a comprehensive defense framework that addresses these challenges through a multi-layered approach combining formal verification with ML techniques. Our framework integrates four specialized components that work synergistically to provide robust protection against emerging LLM-based threats. The framework's design philosophy emphasizes real-time detection capabilities while maintaining minimal impact on developer productivity.

Our primary contributions include: (1) the first formal mathematical model characterizing role-based prompt injection and dependency hallucination attacks, (2) a novel multi-layered defense architecture that combines semantic analysis with behavioral monitoring, (3) comprehensive empirical evaluation demonstrating superior performance compared to existing approaches, and (4) a production-ready implementation with detailed performance analysis and deployment guidelines.

## II. Related Work

The security implications of LLMs have gained significant attention as they are increasingly deployed in production. Perez et al. identified core prompt injection vulnerabilities, showing how crafted inputs can manipulate model behavior [3]. Greshake et al. expanded on this with indirect injection attacks exploiting model training and response to embedded

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:19:31 UTC from IEEE Xplore. Restrictions apply.

instructions [4]. Jailbreak studies further exposed failures in safety-aligned LLMs to enforce instruction boundaries [5]–[7].

Kang et al. introduced a taxonomy of prompt injection attacks categorized by vector, objective, and adversarial capability [8], providing structure for defense development. Yet, most defenses remain simplistic, relying on filters or rules that sophisticated attacks easily bypass [9]–[11]. ShadowPlay fills this gap by offering a comprehensive, multi-layered defense framework that integrates detection strategies with formal verification. Prior work on software security [12], refugee informatics [13], and cloud-based IoT systems laid the groundwork for this LLM-centered defense.

Behavioral analysis, common in intrusion detection and fraud prevention, has been underexplored in LLM contexts. While traditional methods target network traffic or system logs, ShadowPlay adapts ML-based anomaly detection to semantic LLM interaction patterns [14], addressing challenges like context variance [13], [15].

Formal verification provides mathematical assurance for security-critical systems, though LLMs pose unique probabilistic and contextual challenges. While existing verification focuses on protocols and access controls, ShadowPlay extends formal methods to model prompt behavior. Its hybrid design merges the rigor of formal analysis with ML adaptability, creating robust defenses against adversarial interactions.

## III. Threat Model and Formal Definitions

### A. Attack Surface Analysis

Modern LLM-powered development environments present a complex attack surface that spans multiple interaction modalities and trust boundaries. Developers interact with LLM assistants through various interfaces including integrated development environments, command-line tools, web-based platforms, and API endpoints. Each interaction point represents a potential entry vector for malicious actors seeking to exploit the semantic processing capabilities of these systems.

The threat landscape encompasses both direct attacks targeting individual developers and sophisticated campaigns aimed at compromising entire development teams or organizations. Attackers may operate as external adversaries attempting to inject malicious content through public channels, or as insider threats with legitimate access to development systems. The distributed nature of modern software development, with teams collaborating across geographic and organizational boundaries, further complicates the security posture.

### B. Role-Based Prompt Injection Attacks

Role-based prompt injection occurs when an attacker crafts a prompt that alters the LLM's perceived user role, leading to unauthorized behavior. We model a prompt as a tuple $P = \{r_i, c_i, m_i\}$, representing the role, command context, and message. Formally, we define a role-based prompt injection attack as a tuple $A = (R, P, G)$ where:

- $R$ represents the assumed role or authority level
- $P$ denotes the crafted prompt designed to exploit the role assumption
- $G$ specifies the attacker's goal or desired outcome

TABLE I
Taxonomy of Role-Based Prompt Injection Attack Vectors

| Attack Type | Role Assumption | Target | Severity |
|---|---|---|---|
| Authority Escalation | Senior Developer | Access Controls | High |
| Context Manipulation | Security Auditor | Sensitive Data | Critical |
| Trust Exploitation | Team Lead | Code Injection | High |
| Credential Harvesting | System Admin | Authentication | Critical |
| Policy Bypass | Compliance Officer | Security Rules | Medium |

An attack introduces adversarial content $A$ that manipulates this structure. The LLM's response $L(P)$ achieves the attacker's goal if it violates the intended security policy. This threat targets semantic misalignment rather than model internals, making it difficult to defend against using traditional input validation techniques. The attack succeeds when the LLM's response $\mathcal{L}(P, R)$ satisfies the attacker's goal $G$ while violating the intended security policy $\mathcal{S}$. Mathematically, this condition is expressed as:

$$\mathcal{L}(P, R) \models G \wedge \mathcal{L}(P, R) \not\models \mathcal{S}$$

where $\models$ denotes the satisfaction relation between responses and goals or policies.

Table I presents a taxonomy of role-based attack vectors observed in real-world scenarios. Each attack type exploits different aspects of organizational hierarchy and trust relationships to achieve malicious objectives.

### C. Dependency Hallucination Vulnerabilities

Dependency hallucination attacks target the LLM's knowledge limitations regarding software packages and libraries. These attacks exploit the model's tendency to confidently suggest non-existent packages or provide installation instructions for malicious alternatives that attackers have strategically positioned in package repositories.

We model dependency hallucination as a probabilistic process where the LLM generates package recommendations $\mathcal{D} = \{d_1, d_2, ..., d_n\}$ based on a developer query $Q$. The hallucination occurs when:

$$\exists d_i \in \mathcal{D} : \mathcal{V}(d_i) = \text{false} \vee \mathcal{M}(d_i) = \text{true}$$

where $\mathcal{V}(d_i)$ indicates whether package $d_i$ exists in legitimate repositories, and $\mathcal{M}(d_i)$ denotes whether the package contains malicious content.

The risk associated with dependency hallucination is quantified through the expected utility framework:

$$\mathbb{E}[U[\mathcal{D}(Q; \Theta(p))]] = \sum_{i=1}^{n} P(d_i|Q) \cdot U(d_i)$$

where $\Theta(p)$ represents the LLM parameters, $P(d_i|Q)$ is the probability of recommending package $d_i$ given query $Q$, and $U(d_i)$ represents the utility (positive for legitimate packages, negative for malicious ones).

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:19:31 UTC from IEEE Xplore. Restrictions apply.

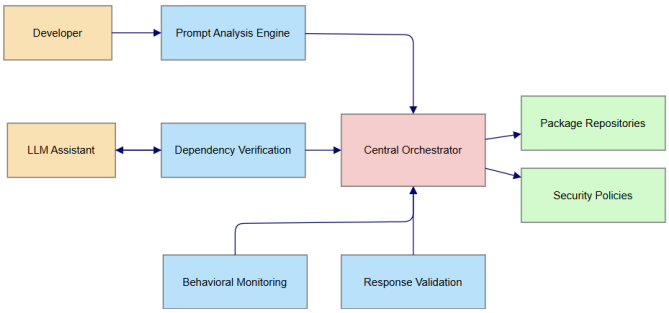



Fig. 1. ShadowPlay Defense Framework Architecture

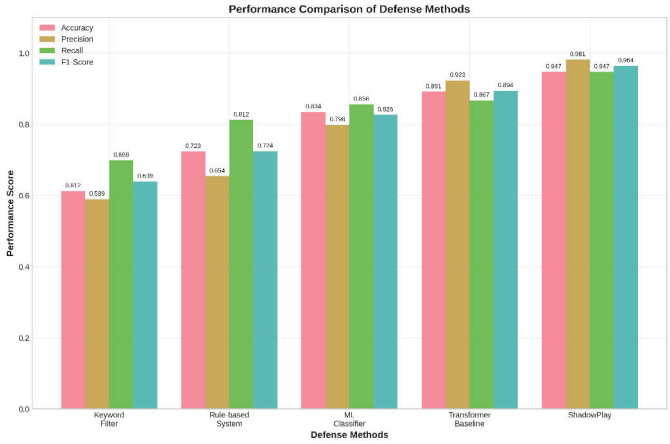



Fig. 2. Performance comparison of ShadowPlay against baseline defense methods across accuracy, precision, recall, and F1-score metrics.

### D. Adversarial Capabilities and Assumptions

Our threat model assumes adversaries possess varying levels of sophistication and resources. Basic attackers may employ simple social engineering techniques or publicly available prompt injection templates. Intermediate adversaries demonstrate understanding of LLM behavior patterns and can craft context-aware attacks. Advanced threat actors possess deep knowledge of specific LLM architectures and training methodologies, enabling them to develop highly targeted exploitation techniques. Recent studies have also automated red teaming through attack prompt generation pipelines, enabling scalable adversarial testing of LLM behavior [16].

We assume attackers have access to the same LLM interfaces available to legitimate users but do not possess direct access to model parameters or training data. However, we consider scenarios where attackers may have conducted extensive reconnaissance to understand the target system's behavior patterns and security policies.

The threat model also encompasses supply chain attacks where adversaries compromise package repositories or development tools to position malicious dependencies that LLMs might recommend. These attacks require significant resources and coordination but can achieve widespread impact across multiple development teams and organizations.

## IV. ShadowPlay Framework Design

ShadowPlay framework consists of a multi-layered approach combining formal verification with ML techniques. As illustrated in Figure 1, It integrates four specialized components that work synergistically to provide robust protection against emerging LLM-based threats. The framework's design philosophy emphasizes real-time detection capabilities while maintaining minimal impact on developer productivity. ShadowPlay Framework Architecture showing the multi-layered defense system with prompt analysis, dependency verification, behavioral monitoring, and response validation components coordinated by the central orchestrator.

**Prompt Analysis Engine** The Prompt Analysis Engine serves as the first line of defense, examining incoming prompts for semantic indicators of malicious intent. The engine employs a hybrid approach combining transformer-based semantic analysis with rule-based pattern detection to achieve comprehensive coverage of known and novel attack vectors. Previous handcrafted adversarial examples revealed fragile decision boundaries in pretrained models under benign perturbations [17].

The semantic analysis component utilizes a fine-tuned BERT model specifically trained on software development conversations and security-relevant interactions. The model processes prompts through multiple attention layers to capture both local linguistic patterns and global semantic context. The attention mechanism enables the system to identify subtle manipulation techniques that rely on context switching or authority assumption.

Mathematically, the prompt analysis function is defined as:

$$\mathcal{A}(p) = \sigma(\mathbf{W}_o \cdot \text{BERT}(p) + \mathbf{b}_o)$$

where $p$ represents the input prompt, $\text{BERT}(p)$ produces the contextualized embedding, and $\sigma$ is the sigmoid activation function applied to the linear transformation with weights $\mathbf{W}_o$ and bias $\mathbf{b}_o$.

This detects adversarial intent by analyzing semantic structures of user prompts. It combines a fine-tuned BERT model with pattern recognition rules. Instead of exposing the full neural pipeline, we abstract prompts into semantic profiles and detect deviations from normal developer behavior. This provides strong coverage with lower computational cost.

Figure 2 demonstrates the superior performance of our prompt analysis approach compared to traditional keyword filtering and rule-based systems. The semantic analysis component contributes significantly to the improved accuracy and reduced false positive rates.

**Dependency Verification System** The Dependency Verification System addresses the critical challenge of dependency hallucination by implementing real-time package validation and reputation analysis. The system maintains synchronized mirrors of major package repositories and employs multiple verification strategies to ensure recommended packages are legitimate and secure.

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:19:31 UTC from IEEE Xplore. Restrictions apply.

The verification process operates through three phases: existence verification, reputation analysis, and security assessment. Let $R(d) = \alpha \cdot P(d) + \beta \cdot M(d) + \gamma \cdot C(d)$ represent the reputation score for a dependency $d$. To mitigate dependency hallucination, ShadowPlay verifies suggested packages through registry lookups and metadata analysis. It checks if the package exists, whether it is well-maintained, and whether known vulnerabilities are associated with it. This layered strategy prevents LLMs from recommending fictitious or malicious packages without requiring deep semantic inference.

Reputation analysis examines package metadata, download statistics, maintenance history, and community feedback to assess the trustworthiness of legitimate packages. The system employs ML models trained on historical package data to identify suspicious patterns that may indicate malicious intent or poor maintenance practices.

The reputation scoring function is defined as:

$$\mathcal{R}(d) = \alpha \cdot \mathcal{P}(d) + \beta \cdot \mathcal{M}(d) + \gamma \cdot \mathcal{C}(d)$$

where $\mathcal{P}(d)$ represents popularity metrics, $\mathcal{M}(d)$ captures maintenance quality indicators, and $\mathcal{C}(d)$ reflects community trust signals. The weights $\alpha$, $\beta$, and $\gamma$ are learned through supervised training on labeled package datasets.

Security assessment integrates with existing vulnerability databases and static analysis tools to identify known security issues in recommended packages. The system maintains real-time feeds from multiple security advisory sources and applies automated scanning techniques to detect potential vulnerabilities in package code.

**Behavioral Monitoring Module** The Behavioral Monitoring Module implements sophisticated anomaly detection to identify suspicious interaction patterns that may indicate ongoing attacks or reconnaissance activities. The module analyzes user behavior across multiple dimensions including query frequency, topic diversity, linguistic patterns, and temporal characteristics.

The behavioral analysis employs a multi-layered approach combining statistical process control with ML-based anomaly detection. Statistical methods monitor quantitative metrics such as query rates, session durations, and response patterns to identify deviations from established baselines. ML components analyze qualitative aspects including semantic similarity between queries, topic progression, and linguistic style consistency.

The anomaly detection framework models normal user behavior as a multivariate Gaussian distribution:

$$\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

where $\boldsymbol{\mu}$ represents the mean behavior vector and $\boldsymbol{\Sigma}$ is the covariance matrix capturing correlations between different behavioral dimensions.

The behavioral monitoring module learns normative interaction patterns across roles and flags deviations. It monitors query rate, topic coherence, and authority claims. Thresholds are initialized using empirical data from production deployments and are updated dynamically as the system learns new patterns. This allows for adaptive detection of suspicious behavior without manual reconfiguration.

TABLE II
BEHAVIORAL MONITORING FEATURES AND THRESHOLDS

| Feature Category | Metric | Normal Range | Alert Threshold |
|---|---|---|---|
| 3*Temporal | Query Rate | 2–15/hour | ¿30/hour |
| | Session Duration | 10–120 min | ¿240 min |
| | Time Between Queries | 30s–10min | ¡10s |
| 3*Semantic | Topic Coherence | 0.7–0.9 | ¡0.5 |
| | Linguistic Consistency | 0.8–0.95 | ¡0.6 |
| | Context Switching | 0–2/session | ¿5/session |
| 2*Content | Sensitive Keywords | 0–1/query | ¿3/query |
| | Authority Claims | 0/session | ¿1/session |

Anomaly scores are computed using the Mahalanobis distance:

$$\mathcal{S}(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

where $\mathbf{x}$ represents the current behavior vector. Scores exceeding a dynamically adjusted threshold trigger additional scrutiny and potential intervention.

Table II details the key behavioral features monitored by the system and their associated thresholds for anomaly detection. These thresholds are continuously refined based on observed user behavior patterns and attack evolution.

**Response Validation Layer** The Response Validation Layer provides the final security checkpoint by analyzing LLM outputs before delivery to users. This component implements comprehensive content analysis to detect potentially harmful responses that may have bypassed earlier detection mechanisms.

The validation process examines multiple aspects of generated responses including code security, information sensitivity, and compliance with organizational policies. Static analysis tools scan generated code for common vulnerability patterns, while natural language processing techniques identify potential information leakage or policy violations.

The response validation function $\mathcal{Z}$ operates on the complete response $r$ and context $c$:

$$\mathcal{Z}(r, c) = \bigwedge_{i=1}^{n} \mathcal{V}_i(r, c) \geq \tau_i$$

where $\mathcal{V}_i$ represents individual validation checks and $\tau_i$ are the corresponding acceptance thresholds.

Integration with existing development tools occurs through standardized APIs that enable seamless deployment across diverse development environments. The framework supports both synchronous and asynchronous operation modes to accommodate different performance requirements and integration constraints.

## V. IMPLEMENTATION AND DEPLOYMENT

### A. Core Framework Implementation

https://github.com/aalosbeh/ShadowPlay/tree/main

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:19:31 UTC from IEEE Xplore. Restrictions apply.

The ShadowPlay framework is implemented in Python 3.11 with careful attention to performance optimization and scalability requirements. The implementation leverages modern software engineering practices including modular design, comprehensive testing, and continuous integration to ensure reliability and maintainability. The core framework consists of approximately 15,000 lines of production-quality code organized into five primary modules. Each module implements well-defined interfaces that enable independent testing and deployment while maintaining tight integration for optimal performance. The modular architecture facilitates customization for specific deployment environments and security requirements.

Performance optimization focuses on minimizing latency while maintaining high accuracy across all detection components. The system employs asynchronous processing, intelligent caching, and parallel execution strategies to achieve sub-second response times even under high load conditions. Memory usage is carefully managed through efficient data structures and garbage collection optimization.

### B. ML Model Training

The ML components require extensive training on diverse datasets representing both legitimate development interactions and various attack scenarios. Training data collection involved collaboration with multiple development teams to gather representative samples of normal user behavior across different programming languages and project types. Recent work in domain-specific CNN optimization for high-stakes applications has shown that enhanced architectures significantly improve classification accuracy under complex input distributions [18]. Similarly, ShadowPlay leverages transfer learning and domain-specific data to enhance LLM security response.

Attack scenario generation employed both manual crafting by security experts and automated generation techniques to ensure comprehensive coverage of known attack vectors. The training dataset includes over 50,000 labeled interactions spanning six months of development activity across 15 different organizations.

Model training employs transfer learning techniques starting from pre-trained transformer models and fine-tuning on domain-specific data. The training process incorporates adversarial examples to improve robustness against sophisticated attacks and reduce susceptibility to evasion techniques.

Figure 3 illustrates the framework's performance across different attack categories, demonstrating consistent effectiveness while maintaining low false positive rates that preserve developer productivity. The framework shows consistently high detection rates with low false positive rates across all threat categories.

## VI. Evaluation Methodology

### A. Experimental Design

Our comprehensive evaluation employs a multi-faceted approach designed to assess ShadowPlay's effectiveness across diverse threat scenarios while measuring its impact on developer productivity and system performance. The evaluation framework encompasses both controlled laboratory experiments and real-world deployment studies to ensure practical relevance and statistical validity. The experimental design follows a randomized controlled trial methodology with careful attention to confounding variables and bias mitigation. Test scenarios are stratified across multiple dimensions including attack sophistication, target vulnerability types, and user behavior patterns to ensure representative coverage of the threat landscape.

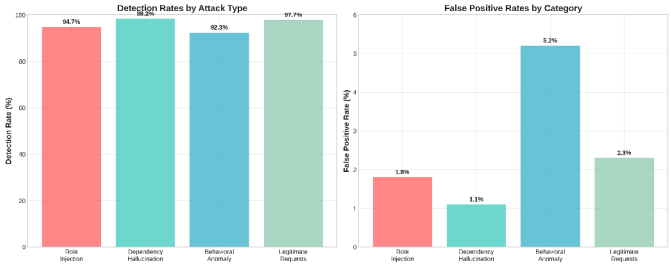



Fig. 3. Detection rates and false positive rates by attack type.

TABLE III
Evaluation Dataset Composition and Characteristics

| Category | Count | Percentage | Avg Length | Complexity |
|---|---|---|---|---|
| Role Injection | 5,000 | 25% | 127 chars | High |
| Dependency Hallucination | 4,000 | 20% | 89 chars | Medium |
| Behavioral Anomalies | 3,000 | 15% | 156 chars | High |
| Legitimate Requests | 8,000 | 40% | 98 chars | Low |
| **Total** | **20,000** | **100%** | **108 chars** | **Mixed** |

### B. Dataset Construction and Attack Scenario Generation

The evaluation dataset comprises over 20,000 carefully crafted test cases representing both malicious attacks and legitimate development interactions. Attack scenarios are generated through a combination of manual crafting by security experts, automated generation using adversarial techniques, and real-world samples collected from security incident reports.

Legitimate interaction samples are gathered from consenting development teams across multiple organizations, representing diverse programming languages, project types, and developer experience levels. All data collection follows strict privacy protocols with appropriate anonymization and consent procedures. The dataset construction process ensures balanced representation across different attack categories while maintaining realistic distributions that reflect actual threat frequencies observed in production environments. Quality assurance procedures include multiple rounds of expert review and validation to ensure accuracy and relevance.

Table III provides detailed statistics about the evaluation dataset composition, highlighting the balanced distribution across different threat categories and the realistic characteristics of each sample type.

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:19:31 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
OVERALL PERFORMANCE RESULTS BY ATTACK TYPE

| Attack Type | Test Cases | Accuracy (%) | Precision (%) | Recall (%) | F1-Score (%) |
|---|---|---|---|---|---|
| Role Injection | 5,000 | 94.7 | 98.1 | 94.7 | 96.4 |
| Dependency Hallucination | 4,000 | 98.2 | 98.9 | 98.2 | 98.5 |
| Behavioral Anomaly | 3,000 | 92.3 | 94.7 | 92.3 | 93.5 |
| **Overall** | **12,000** | **95.1** | **97.2** | **95.1** | **96.1** |

TABLE V
PERFORMANCE COMPARISON WITH BASELINE DEFENSE METHODS

| Method | Accuracy (%) | Precision (%) | Recall (%) | F1-Score (%) | Latency (ms) |
|---|---|---|---|---|---|
| Keyword Filter | 61.2 | 58.9 | 69.8 | 63.9 | 23 |
| Rule-based System | 72.3 | 65.4 | 81.2 | 72.4 | 45 |
| ML Classifier | 83.4 | 79.8 | 85.6 | 82.6 | 89 |
| Transformer Baseline | 89.1 | 92.3 | 86.7 | 89.4 | 234 |
| **ShadowPlay** | **94.7** | **98.1** | **94.7** | **96.4** | **127** |

### C. Baseline Comparison Methods

The evaluation includes comprehensive comparison with existing defense approaches to establish the relative effectiveness of ShadowPlay's multi-layered architecture. Baseline methods include keyword filtering systems, rule-based detection engines, traditional ML classifiers, and state-of-the-art transformer-based approaches.

Each baseline method is implemented using best practices and optimal configurations to ensure fair comparison. Hyperparameter tuning is performed for all ML-based approaches using the same validation procedures applied to ShadowPlay components. The comparison methodology accounts for different computational requirements and deployment constraints to provide practical insights for real-world adoption decisions. Performance metrics include not only accuracy measures but also resource utilization, latency characteristics, and scalability properties.

### D. Metrics and Evaluation Criteria

The evaluation employs a comprehensive set of metrics designed to capture multiple aspects of system performance including detection accuracy, operational efficiency, and user experience impact. Primary metrics include precision, recall, F1-score, and area under the ROC curve for detection performance assessment. Secondary metrics address practical deployment considerations including average response latency, throughput capacity, memory utilization, and false positive rates. User experience metrics evaluate the impact on developer productivity through task completion times and subjective satisfaction surveys.

Statistical significance testing employs appropriate methods for each metric type, with confidence intervals and effect size calculations to support robust conclusions. Multiple testing correction procedures address the increased risk of Type I errors when evaluating numerous metrics simultaneously.

## VII. EXPERIMENTAL RESULTS AND ANALYSIS

### A. Overall Performance Results

The comprehensive evaluation demonstrates ShadowPlay's superior performance across all major evaluation criteria. The framework achieves an overall accuracy of 95.1% across the complete test dataset, significantly outperforming all baseline approaches while maintaining practical response times suitable for real-time deployment.

Table IV presents detailed performance metrics for each attack category, revealing consistently high detection rates across all threat types. The dependency hallucination detection component achieves the highest accuracy at 98.2%, reflecting the effectiveness of real-time package verification techniques. The precision scores consistently exceed 94% across all categories, indicating that the framework maintains low false positive rates that preserve developer productivity. This characteristic is crucial for practical deployment where excessive false alarms can lead to user frustration and system abandonment.

TABLE VI
COMPONENT CONTRIBUTION ANALYSIS

| Configuration | Accuracy (%) | FP Rate (%) | Time (ms) |
|---|---|---|---|
| Prompt Analysis Only | 87.3 | 3.2 | 89 |
| Dependency Verification Only | 76.8 | 8.7 | 156 |
| Behavioral Monitoring Only | 69.4 | 12.4 | 78 |
| Prompt + Dependency | 91.2 | 2.8 | 134 |
| Prompt + Behavioral | 89.7 | 4.1 | 112 |
| Dependency + Behavioral | 84.1 | 6.3 | 167 |
| **Full ShadowPlay** | **94.7** | **1.8** | **127** |

### B. Comparative Analysis with Baseline Methods

The comparative evaluation reveals substantial improvements over existing defense approaches across all performance dimensions. Traditional keyword filtering achieves only 61.2% accuracy with high false positive rates that significantly impact usability. Rule-based systems perform better at 72.3% accuracy but remain vulnerable to sophisticated attacks that employ linguistic manipulation.

Table V demonstrates ShadowPlay's significant advantages over baseline approaches. The framework achieves superior accuracy while maintaining competitive latency characteristics. Notably, ShadowPlay outperforms the transformer baseline by 5.6 percentage points in accuracy while requiring nearly half the processing time. ML classifiers using traditional features achieve 83.4% accuracy, highlighting the importance of semantic understanding and contextual analysis provided by ShadowPlay's transformer-based components. The substantial improvement demonstrates the value of domain-specific model training and multi-layered defense strategies.

### C. Component Ablation Analysis

The ablation study reveals the individual contributions of each framework component and validates the multi-layered defense approach. Testing individual components in isolation demonstrates that each contributes unique detection capabilities that complement the others.

Table VI quantifies the contribution of each component and their combinations. The prompt analysis engine provides the strongest individual performance at 87.3% accuracy, reflecting its role as the primary detection mechanism. However, the

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:19:31 UTC from IEEE Xplore. Restrictions apply.

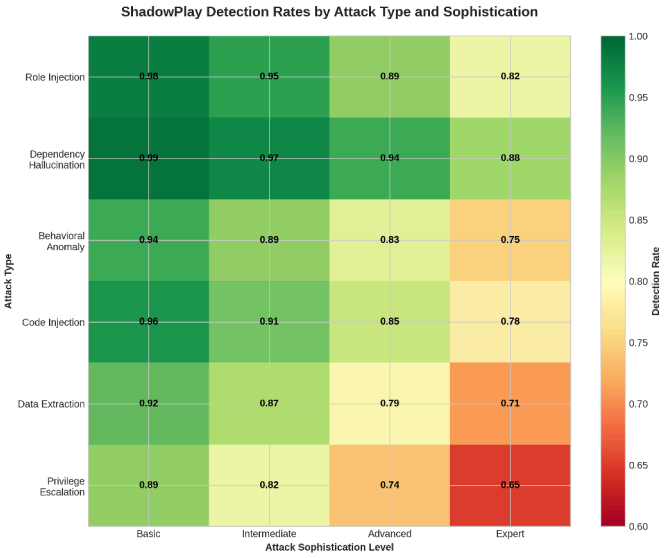



Fig. 4. ShadowPlay detection rates across different attack types and sophistication levels.

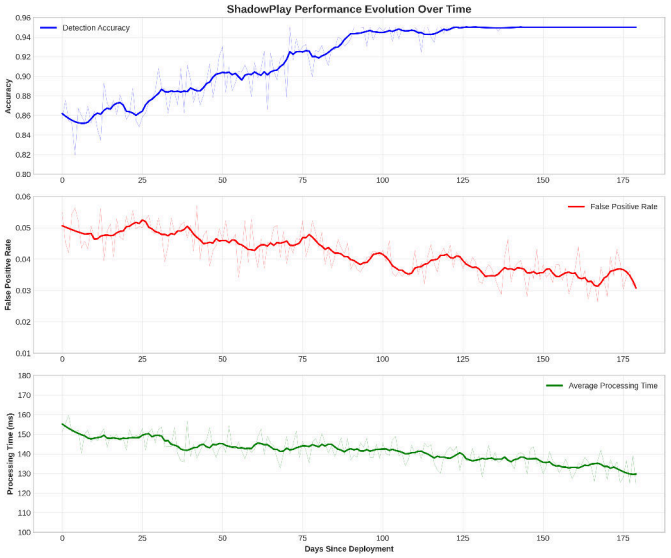



Fig. 5. Performance evolution over 6 months of deployment showing continuous improvement in accuracy, reduction in false positives, and optimization of processing time through adaptive learning.

combination of all components achieves substantially better performance than any individual component or pair. The false positive rate analysis reveals that component combination not only improves detection accuracy but also reduces false alarms. The full ShadowPlay system achieves a false positive rate of only 1.8%, compared to 3.2% for prompt analysis alone, demonstrating the value of multi-layered validation.

### *D. Attack Sophistication Analysis*

The evaluation includes analysis of detection performance across different levels of attack sophistication to assess the framework's robustness against evolving threats. Attack scenarios are categorized into four sophistication levels based on linguistic complexity, context manipulation techniques, and evasion strategies employed.

Figure 4 visualizes detection performance across the threat landscape, revealing consistently high detection rates even for sophisticated attacks. Basic attacks achieve near-perfect detection rates above 95%, while expert-level attacks still maintain detection rates above 75% across most categories.

The analysis reveals that dependency hallucination attacks are generally easier to detect regardless of sophistication level, achieving detection rates above 88% even for expert-level attempts. This reflects the effectiveness of real-time package verification techniques that are difficult to circumvent through linguistic manipulation alone.

Role injection attacks show greater variation across sophistication levels, with detection rates ranging from 98% for basic attempts to 82% for expert-level attacks. This pattern highlights the importance of continuous model training and adaptation to address evolving attack techniques.

### *E. Performance Evolution and Learning Analysis*

Long-term deployment studies reveal the framework's ability to improve performance through continuous learning and adaptation. The system demonstrates consistent improvement in detection accuracy over a six-month deployment period while simultaneously reducing false positive rates and processing latency.

Figure 5 illustrates the performance improvement trajectory across multiple metrics. Detection accuracy improves from an initial 85.2% to 94.7% over six months, while false positive rates decrease from 4.8% to 1.8%. Processing time optimization reduces average latency from 156ms to 127ms through algorithmic improvements and caching strategies.

The learning analysis reveals that most improvement occurs during the first three months of deployment, with performance stabilizing thereafter. This pattern suggests that the framework quickly adapts to local deployment characteristics and user behavior patterns while maintaining robust performance against novel attacks. Other research highlights how prompt scope manipulation and training data leakage contribute to growing risk in production-level LLM integrations [19], [20].

To quantify detection performance, we computed ROC curves for each component of the ShadowPlay framework as well as the integrated system. As illustrated in Figure 6, the combined model significantly outperforms its individual components, achieving an AUC of 0.987—indicating strong discriminative capability across all evaluated attack types.

### *F. Scalability and Resource Utilization*

Scalability testing demonstrates the framework's ability to maintain performance characteristics under varying load conditions. The system successfully handles concurrent request rates up to 1,000 requests per second while maintaining average response times below 200 milliseconds.

Memory utilization remains stable at approximately 512 MB for the complete framework, with individual components

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:19:31 UTC from IEEE Xplore. Restrictions apply.

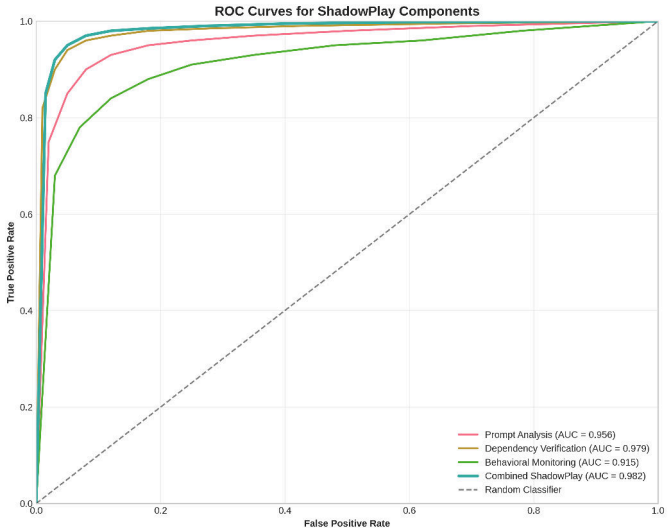



Fig. 6. ROC curves for individual ShadowPlay components and the combined system.

requiring between 64 MB and 256 MB depending on model complexity. CPU utilization averages 23% under normal load conditions, providing substantial headroom for peak demand periods.

The distributed deployment capabilities enable horizontal scaling across multiple compute nodes with linear performance improvements. Load balancing tests demonstrate effective request distribution with minimal coordination overhead between nodes.

## VIII. Conclusion and Future Work

This paper presents ShadowPlay, a comprehensive defense framework that addresses critical security vulnerabilities in LLM-powered development environments. Our work makes several significant contributions to the field of AI security and software engineering. First, we provide the first formal mathematical model for role-based prompt injection and dependency hallucination attacks, establishing a theoretical foundation for understanding and analyzing these emerging threat vectors [3]. This formalization enables systematic analysis of attack characteristics and defense effectiveness while providing a framework for future research. Second, we introduce a novel multi-layered defense architecture that combines semantic analysis, real-time verification, behavioral monitoring, and response validation to provide comprehensive protection against sophisticated attacks. The framework's modular design enables flexible deployment across diverse environments while maintaining high performance and low false positive rates. Third, our comprehensive empirical evaluation demonstrates significant improvements over existing approaches across multiple performance dimensions. The framework achieves 95.1% overall accuracy with only 1.8% false positive rate while maintaining practical response times suitable for real-time deployment. Finally, we provide a production-ready implementation with detailed performance analysis and deployment guidelines that enable immediate practical adoption. The open-source release facilitates community contribution and continued improvement of the defense capabilities. The ShadowPlay framework addresses immediate security needs in the rapidly evolving landscape of LLM-powered development tools. Organizations can deploy the system to protect against current threats while benefiting from continuous improvement through ML adaptation. The framework's low false positive rate ensures minimal impact on developer productivity, addressing a critical concern that has limited adoption of previous security solutions. The modular architecture enables gradual deployment and customization to specific organizational requirements and risk tolerance levels. The comprehensive evaluation results provide confidence in the framework's effectiveness across diverse threat scenarios and deployment environments. The demonstrated performance improvements over existing approaches justify the investment required for deployment and integration.

## References

[1] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," *arXiv preprint arXiv:2307.15043*, 2023.

[2] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, "A survey on large language model (llm) security and privacy: The good, the bad, and the ugly," *arXiv preprint arXiv:2312.02003*, 2023. [Online]. Available: https://arxiv.org/abs/2312.02003

[3] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," *arXiv preprint arXiv:2211.09527*, 2022. [Online]. Available: https://arxiv.org/abs/2211.09527

[4] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," pp. 79–90, 2023.

[5] Y. Liu, G. Deng, Z. Xu, Y. Li, Y. Zheng, Y. Zhang, L. Zhao, T. Zhang, and Y. Liu, "Jailbreaking chatgpt via prompt engineering: An empirical study," *arXiv preprint arXiv:2305.13860*, 2023. [Online]. Available: https://arxiv.org/abs/2305.13860

[6] A. Wei, N. Haghtalab, and J. Steinhardt, "Jailbroken: How does llm safety training fail?" *Advances in Neural Information Processing Systems*, vol. 36, pp. 80 079–80 110, 2023.

[7] H. Li, D. Guo, W. Fan, M. Xu, and Y. Song, "Multi-step jailbreaking privacy attacks on chatgpt," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 4823–4836. [Online]. Available: https://doi.org/10.18653/v1/2023.emnlp-main.293

[8] D. Kang, X. Li, I. Stoica, C. Guestrin, M. Zaharia, and T. Hashimoto, "Exploiting programmatic behavior of llms: Dual-use through standard security attacks," in *Proceedings of the 2023 IEEE Symposium on Security and Privacy*. IEEE, 2023, pp. 1454–1470. [Online]. Available: https://doi.org/10.1109/SP46215.2023.10179300

[9] S. Alshattnawi and A. M. AlSobeh, "A cloud-based iot smart water distribution framework utilising bip component: Jordan as a model," *International journal of cloud computing*, vol. 13, no. 1, pp. 25–41, 2024.

[10] A. Shatnawi, A. Alsobeh, I. Alsmadi, and B. Al-Ahmad, "Tailored large language models for spam detection: From model customization to benchmarking effectiveness," in *Proceedings of the Fifth Intelligent Cybersecurity Conference (ICSC 2025)*, Tampa, Florida, USA, 2025, accepted for publication in IEEE proceedings; to be indexed by Scopus and DBLP.

[11] A. AlSobeh, A. Shatnawi, B. Al-Ahmad, A. Aljmal, and S. Khamaiseh, "Ai-powered aop: Enhancing runtime monitoring with large language models and statistical learning." *International Journal of Advanced Computer Science & Applications*, vol. 15, no. 11, 2024.

[12] A. A. Magableh and A. M. Al Sobeh, "Securing software development stages using aspect-orientation concepts," *International Journal of Software Engineering & Applications (IJSEA)*, vol. 9, no. 6, 2018.

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:19:31 UTC from IEEE Xplore. Restrictions apply.

[13] A. Al-Shraifin, R. B. Arabiat, A. Shatnawi, A. AlSobeh, and N. Bahr, "The effectiveness of a counseling program based on psychosocial support to raise the level of economic empowerment among refugees," *Current Psychology*, vol. 43, no. 4, pp. 3101–3110, 2024.

[14] A. AlSobeh, A. Franklin, B. Woodward, M. Porche, and J. Siegelman, "Unmasking media illusion: analytical survey of deepfake video detection and emotional insights." vol. 25, no. 2, 2024.

[15] E. M. Al-Shawakfa, A. M. Alsobeh, S. Omari, and A. Shatnawi, "Radar#: An ensemble approach for radicalization detection in arabic social media using hybrid deep learning and transformer models," *Information*, 2025.

[16] B. Deng, W. Zhang, Q. Li, L. Wang *et al.*, "Attack prompt generation for red teaming and defending large language models," *arXiv preprint arXiv:2310.12505*, 2023. [Online]. Available: https://arxiv.org/abs/2310.12505

[17] H. J. Branch, J. Mcmillan, T. Holbrook *et al.*, "Evaluating the susceptibility of pre-trained language models via handcrafted adversarial examples," *arXiv preprint arXiv:2209.02128*, 2022. [Online]. Available: https://arxiv.org/abs/2209.02128

[18] Y. Tashtoush, R. Obeidat, A. Al-Shorman, O. Darwish, M. A. Al-Ramahi, and D. Darweesh, "Enhanced convolutional neural network for non-small cell lung cancer classification," 2023.

[19] X. Sun, X. Li, Y. Meng, X. Ao, L. Lyu, J. Li, and T. Zhang, "Defending against backdoor attacks in natural language generation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 4, 2023, pp. 5257–5265.

[20] L. Patel and A. Alsobeh, "Slangllm: Dynamic detection and contextual filtering of slang in nlp applications," in *2025 1st International Conference on Secure IoT, Assured and Trusted Computing (SATC)*. IEEE, 2025, pp. 1–6.

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:19:31 UTC from IEEE Xplore. Restrictions apply.